```
    var cities = await service.GetAllAsync();                    ❷
    return new SelectList(cities, nameof(City.Id),
    ➥ nameof(City.Name), null, "Country.CountryName");
}
```

❶ You amend the handler methods and the GetCityOptions methods to make them asynchronous.

❷ Obtain the data from the SimpleCityService class instead of generating it in the PageModel.

Aside from the conversion to using asynchronous methods, the only real change here is that the responsibility for generating the collection of cities is no longer that of the `CreateModel` class. That job has been delegated to the new class: the `SimpleCityService`. The `CreateModel` class *depends* on the `SimpleCityService` for the data, making the `SimpleCityService` a *dependency* of the `CreateModel` class.

7.1.2 Loose coupling

I believe it was Steve Smith, aka "Ardalis," a well-known ASP.NET speaker, author, and trainer, who first coined the phrase "new is glue" (https://ardalis.com/new-is-glue/). He suggests that whenever you use the `new` keyword in C# code, you consider whether you are creating a *tight coupling* between a consumer (a "high-level module") and its dependencies ("low-level modules") and, if so, what ramifications that might have in the longer term. In this example, while shifting the logic for providing city data into its own component, you have effectively glued your `SimpleCityService component` to the `CreateModel` class. The relationship between these actors is shown in figure 7.2.